Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

Thème principal : Algorithmique fondamentale : Structures de contrôle itératives (POUR, TANT QUE, RÉPÉTER... JUSQU'À), variants et invariants de boucle

Synthèse exécutive : Ce cours pose les bases indispensables de l'itération en programmation structurée. L'objectif est d'éliminer la duplication de code pour traiter des ensembles de données de taille arbitraire. Il formalise le choix rigoureux entre boucles bornées (nombre de tours fixé en amont) et non bornées (arrêt sur condition dynamique), tout en fournissant les outils mathématiques d'assurance qualité logicielle : le 'variant de boucle' pour démontrer formellement l'absence de boucle infinie (terminaison), et 'l'invariant de boucle' pour garantir mathématiquement l'exactitude du résultat final produit (correction partielle).

Pourquoi itérer ? Typologie des trois structures répétitives fondamentales

En algorithmique, une structure de contrôle itérative permet d'exécuter un bloc d'instructions de 0 à N fois sans recopier textuellement ce bloc dans le code source. On distingue deux grandes familles : l'itération bornée et l'itération non bornée. Dans une boucle 'POUR', l'indice progresse d'une borne de départ à une borne d'arrivée avec un pas fixe, ce qui signifie que l'ordinateur connaît d'avance le nombre d'itérations. À l'inverse, les boucles 'TANT QUE' et 'RÉPÉTER... JUSQU'À' répondent à des situations où l'arrêt dépend de l'évolution dynamique de données en mémoire (ex: lecture de fichier, saisie utilisateur, convergence numérique). Le choix entre 'TANT QUE' et 'RÉPÉTER' repose exclusivement sur le fait que le traitement doive obligatoirement être appliqué au moins une fois (cas typique du menu interactif ou du contrôle de saisie) ou puisse être complètement ignoré si les conditions ne sont pas réunies au départ.

- Anti-pattern du Copier-Coller : Dupliquer du code le rend rigide, illisible et impossible à maintenir face à des volumes de données variables.
- Boucle Bornée (POUR) : Utilisée lorsque le nombre précis d'itérations est connu avant d'entrer dans la boucle. Elle utilise une variable compteur incrémentée automatiquement.
- Boucle Non Bornée à Pré-condition (TANT QUE) : La condition logique est évaluée avant chaque tour ; si elle est fausse dès l'initialisation, le corps s'exécute 0 fois.
- Boucle Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le corps s'exécute d'abord, puis la condition d'arrêt est testée ; garantit au moins 1 exécution.

Cas concret enseignant : *L'analogie du coureur sur piste d'athlétisme : si le coach ordonne 'Fais 10 tours', il s'agit d'une boucle POUR (nombre déterminé). S'il ordonne 'Cours tant qu'il ne pleut pas', c'est un TANT QUE : s'il pleut déjà au vestiaire, le coureur ne commence même pas (0 tour possible). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À car le coureur est obligé de réaliser au minimum un premier tour avant d'évaluer son essoufflement.*

Preuve formelle de Terminaison : Le Variant de Boucle

Pour démontrer qu'une boucle non bornée ne tournera pas indéfiniment, l'informaticien doit formaliser un 'variant de boucle'. Un variant V est une quantité à valeurs entières positives qui dépend des variables modifiées dans la boucle. Il doit satisfaire deux propriétés mathématiques strictes : premièrement, à chaque itération k, $V_{k+1} < V_k$ (décroissance stricte d'au moins 1) ; deuxièmement, dès que V atteint un seuil minimal fixé (minorant), la condition de boucle devient fausse. L'identification explicite d'un variant valide constitue la preuve scientifique irréfutable que l'algorithme s'arrête en un nombre fini d'étapes.

- Boucle Infinie : Dysfonctionnement critique bloquant le fil d'exécution du processeur lorsque la condition d'arrêt n'est jamais atteinte.
- Variant de Boucle : Fonction entière associée à l'état du programme qui décroît strictement à chaque passage.

• Borne Inférieure (Minoration) : Valeur plancher (souvent 0) sous laquelle le variant ne peut descendre sans violer la condition de boucle.

• Preuve de Terminaison : Toute suite d'entiers strictement décroissante et minorée est nécessairement finie, garantissant l'arrêt du programme.

Cas concret enseignant : *La métaphore de la jauge d'essence d'une voiture : à chaque kilomètre parcouru (itération), le volume de carburant restant diminue obligatoirement (décroissance stricte). Comme le réservoir ne peut pas descendre sous 0 litre (minorant), le véhicule finira obligatoirement par s'immobiliser. Si la jauge ne baisse pas ou remonte, le véhicule tourne à l'infini : c'est le bug de la boucle infinie.*

Preuve formelle de Correction : L'Invariant de Boucle

L'invariant de boucle est l'analogue du raisonnement par récurrence pour les algorithmes. Il ne sert pas à prouver l'arrêt (rôle du variant), mais à prouver que le programme calcule exactement ce qui est spécifié dans le cahier des charges sans introduire d'erreur hors-limite ou d'accumulation fausse. À la sortie de la boucle, l'invariant reste vrai et la condition de continuation est devenue fausse. C'est l'intersection de ces deux vérités logiques qui démontre la correction totale ou partielle de l'algorithme.

• Invariant de Boucle : Prédicat logique (assertion) restant rigoureusement VRAI avant l'entrée dans la boucle et à la fin de chaque tour.

• Phase d'Initialisation : Démonstration que l'invariant est vrai dès l'état initial des variables, avant le premier tour.

• Phase de Conservation (Hérédité) : Démonstration que si l'invariant est vrai au début d'une itération arbitraire, il demeure vrai à la fin de celle-ci après modification des variables.

• Phase de Conclusion (Correction) : Combinaison logique de l'invariant final et de la sortie de boucle, établissant que le résultat calculé est mathématiquement correct.

Cas concret enseignant : *L'exemple du sac du livreur de journaux : au départ de sa tournée matinale, l'égalité 'Nombre de journaux distribués + Nombre de journaux restants dans le sac = Nombre total initial' est vraie. À chaque maison livrée, les deux nombres changent mais leur somme reste rigoureusement invariante. À la fin, quand le sac est vide (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut systématiquement remplacer une boucle 'POUR' par une boucle 'TANT QUE' ?
Réponse du professeur : Oui, absolument. Toute boucle POUR peut être simulée par une boucle TANT QUE en déclarant manuellement un compteur avant la boucle, en testant la borne supérieure dans la condition, et en incrémentant soi-même le compteur à la fin du corps. En revanche, l'inverse est faux : on ne peut pas utiliser une boucle POUR pour une saisie interactive ou une lecture de flux tant qu'on ne connaît pas à l'avance le moment où l'utilisateur décidera de s'arrêter.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que les instructions s'exécutent si i est déjà égal à N dès le début ?
Réponse du professeur : Non, aucune instruction du corps ne sera exécutée. Le TANT QUE est une structure à pré-condition : le test logique est systématiquement évalué avant d'autoriser l'accès au corps de boucle. Puisque i < N est faux dès le début, la machine saute immédiatement après le 'FinTantQue' (0 tour effectué).

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique le calcul de la somme des N premiers entiers pairs stricts (par exemple pour N=3 : 2 + 4 + 6 = 12). Justifier le choix de la boucle et expliciter le variant de boucle garantissant la terminaison.

Résolution pas à pas :
Étape 1 : Choix de structure. Le paramètre N est fourni au départ ; le nombre total d'itérations est donc parfaitement déterminé et borné. La boucle 'POUR' est la structure la plus adaptée et la plus sûre. Étape 2 : Définition des

variables. Nous avons besoin de 'somme' (accumulateur initialisé à 0), de 'terme' (premier entier pair strict, soit 2), et d'une variable de contrôle 'i' variant de 1 à N. Étape 3 : Écriture du pseudo-code formel : Algorithme SommePairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant. Si l'on réécrit ce parcours sous forme de boucle générale avec compteur i, le variant de boucle est l'expression V = (N - i + 1). À chaque passage, i augmente de 1, donc V diminue strictement de 1. Lorsque V atteint 0, la boucle s'arrête, ce qui garantit la terminaison.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle : Affichage de 12. L'algorithme a bien sommé les 3 premiers entiers pairs stricts en exactement N itérations avec une terminaison formellement prouvée.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale en mémoire et à l'exécution entre une boucle 'TANT QUE' et une boucle 'RÉPÉTER... JUSQU'À' ?

R : La boucle 'TANT QUE' teste la condition AVANT d'entrer dans le corps (0 itération possible si la condition est fausse dès le début). La boucle 'RÉPÉTER... JUSQU'À' teste la condition APRÈS avoir exécuté les instructions (au moins 1 itération garantie dans tous les cas).

Q : Quelles sont les deux propriétés obligatoires d'un variant de boucle pour prouver la terminaison d'un algorithme ?

R : Le variant doit être une valeur entière strictement décroissante à chaque itération (diminution d'au moins 1 par tour) et être minoré par une constante finie (souvent 0), rendant une exécution infinie mathématiquement impossible.

Q : Quel oubli classique dans une boucle TANT QUE entraîne systématiquement un 0 sur la question en partiel de R1.01 ?

R : L'oubli de la modification de la variable de contrôle dans le corps de la boucle (par exemple oublier 'i <- i + 1'), ce qui rend la condition perpétuellement vraie et crée une boucle infinie qui fige le programme.